

# Machine Learning

## Supervised Learning

Labeled Data

### Classification

1. Logistic Regression
2. Naive Bayes Classifier
3. K-Nearest Neighbors
4. Support Vector Machine
5. Decision Trees
6. Random Forest
7. Gradient Boosting (XGBoost, LightGBM)
8. Neural Networks

#### Example

1. Email spam detection
2. Image classification

### Regression

1. Linear Regression
2. Ridge Regression
3. Lasso Regression
4. Elastic Net Regression
5. Support Vector Regression
6. Random Forest Regression
7. Gradient Boosting Regression

#### Example

1. Stock market prediction
2. Rainfall prediction

## Unsupervised Learning

Unlabeled Data

### Clustering

1. K-means
2. K-medians
3. Hierarchical Clustering
4. DBSCAN
5. Gaussian Mixture Models (GMM)

#### Example

1. Customer segmentation
2. Document clustering

### Association Analysis

1. APRIORI
2. Eclat
3. FP-Growth

#### Example

1. Market basket analysis

### Dimensionality Reduction

1. Feature Extraction
  - Principal Component Analysis (PCA)
  - Autoencoders
2. Feature Selection
  - Filter, Wrapper & Embedded Methods

#### Example

1. Analysis of written texts
2. DNA microarray data analysis

## Reinforcement Learning

State and Action

### Model-Free

1. Q-Learning
2. Deep Q-Network (DQN)
3. Policy Optimization (PPO)
4. Actor-Critic Methods

### Model-Based

1. Learn a model of the environment
2. Plan using the learned model

#### Example

1. Autonomous driving
2. Robotics planning/control
3. Game playing

### Multi-Agent System

1. Multi-Agent System
2. Motion Planning
3. Navigation

## What's Important in 2026?

- Deep Learning & Foundation Models
- Generative AI & Multimodal Models
- LLMs, RAG & AI Agents
- AutoML, MLOps & AI Engineering

# Essential Machine Learning Algorithms

A quick guide to the most important ML algorithms, how they work, and when to use them.

| Algorithm                                                                              | Key Idea / Formula                                                                                                                                                                                                                                                                                   | Illustration | Use Cases                                                                                                                                                                                                      |
|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Linear Regression</b><br>(Supervised – Regression)                                  | Predicts a continuous value using a linear relationship.<br>$\hat{y} = w^T x + b$                                                                                                                                                                                                                    |              | <ul style="list-style-type: none"><li>Predict house prices</li><li>Sales forecasting</li><li>Any continuous value prediction</li></ul>                                                                         |
| <b>Logistic Regression</b><br>(Supervised – Classification)                            | Models the probability of a class using the sigmoid function.<br>$P(y = 1) = \frac{1}{1 + e^{-(w^T x + b)}}$                                                                                                                                                                                         |              | <ul style="list-style-type: none"><li>Binary classification (spam detection)</li><li>Medical risk prediction</li><li>Churn prediction</li></ul>                                                                |
| <b>Decision Tree</b><br>(Supervised – Classification/Regression)                       | Splits data based on feature values to make decisions.                                                                                                                                                                                                                                               |              | <ul style="list-style-type: none"><li>Customer classification</li><li>Churn prediction</li><li>Credit risk assessment</li><li>Works for both classification and regression</li></ul>                           |
| <b>Random Forest</b><br>(Supervised – Classification/Regression)                       | Combines many decision trees and aggregates their predictions. <ul style="list-style-type: none"><li>For classification: <math>\hat{y} = \text{majority vote}(h_1(x), \dots, h_N(x))</math></li><li>For regression: <math display="block">\hat{y} = \frac{1}{N} \sum_{i=1}^N h_i(x)</math></li></ul> |              | <ul style="list-style-type: none"><li>Often strong predictive performance</li><li>Can reduce overfitting compared to a single tree</li><li>Works well on complex datasets</li><li>Feature importance</li></ul> |
| <b>Support Vector Machine (SVM)</b><br>(Supervised – Classification/Regression)        | Finds a decision boundary with maximum margin for classification. For regression (SVR), it fits a function within an $\epsilon$ -insensitive margin.                                                                                                                                                 |              | <ul style="list-style-type: none"><li>Text classification</li><li>Image classification</li><li>Works well with high-dimensional data</li><li>Effective with clear margins</li></ul>                            |
| <b>K-Nearest Neighbors (KNN)</b><br>(Supervised – Classification/Regression)           | Predicts using the majority class or average value of the $k$ nearest data points.                                                                                                                                                                                                                   |              | <ul style="list-style-type: none"><li>Simple and easy to understand</li><li>Works well on small datasets</li><li>Used for both classification and regression</li></ul>                                         |
| <b>K-Means Clustering</b><br>(Unsupervised – Clustering)                               | Groups similar data points into $k$ clusters by minimizing the within-cluster sum of squared distances to cluster centroids.<br>$\min \sum_{i=1}^k \sum_{x \in C_i} \ x - \mu_i\ ^2$                                                                                                                 |              | <ul style="list-style-type: none"><li>Customer segmentation</li><li>Image compression</li><li>Grouping similar items</li><li>Exploratory data analysis</li></ul>                                               |
| <b>Principal Component Analysis (PCA)</b><br>(Unsupervised – Dimensionality Reduction) | Transforms data to a lower dimensional space while preserving maximum variance.                                                                                                                                                                                                                      |              | <ul style="list-style-type: none"><li>Data visualization</li><li>Noise reduction</li><li>Preprocessing for other algorithms</li><li>Handling high-dimensional data</li></ul>                                   |

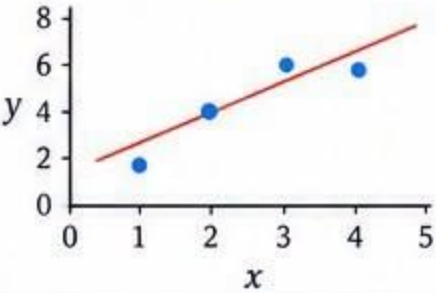
## Python Implementation (Scikit-learn)

```
import numpy as np
from sklearn.linear_model import LinearRegression

# Linear Regression
X = np.array([[1], [2], [3], [4]])
y = np.array([2, 4, 5, 7])
model = LinearRegression()
model.fit(X, y)
print('Slope:', model.coef_[0])
print('Intercept:', model.intercept_)
```

## Example Output

For the above data:  
Slope: 1.6  
Intercept: 0.5

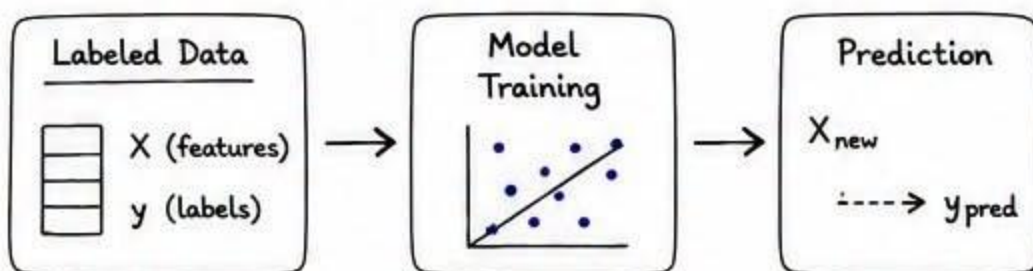


## Tips

- 1 Start with a simple algorithm (e.g., Linear Regression, KNN).
- 2 Try multiple algorithms and compare performance.
- 3 Use proper data preprocessing (scaling, encoding).
- 4 Evaluate with metrics like accuracy, precision, recall or RMSE.
- 5 There is no best algorithm – it depends on your data and problem.
- 6 Always validate your results with a test set or cross-validation.

# 1. SUPERVISED LEARNING

Learn from labeled data to predict the output.



## Common Algorithms

### Classification

- Logistic Regression
- K-Nearest Neighbors
- Support Vector Classifier (SVC)
- Decision Tree
- Random Forest
- Naive Bayes

### Regression

- Linear Regression
- Ridge Regression / Lasso Regression
- Decision Tree Regressor
- Random Forest Regressor
- Gradient Boosting
- Support Vector Regression (SVR)

Goal: Learn mapping from inputs (X) to outputs (y).

## Python Code Example

```
# Import
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Sample data
X = [[1, 2], [2, 3], [3, 4], [4, 5]]
y = [0, 0, 1, 1]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)

# Model
model = RandomForestClassifier(random_state=42)

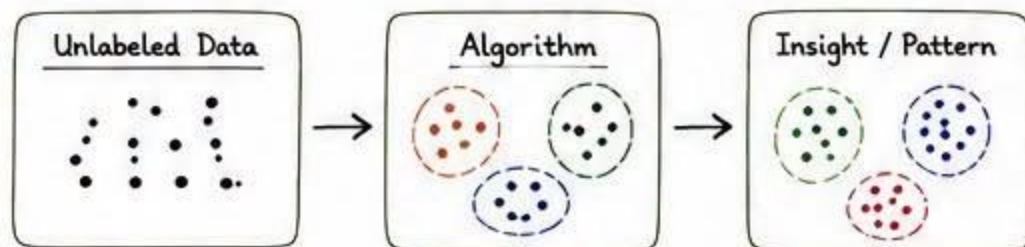
# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate
print("Accuracy:", accuracy_score(y_test, y_pred))
```

# 2. UNSUPERVISED LEARNING

Find patterns or structure in unlabeled data.



## Common Algorithms

### Clustering

- K-Means
- Hierarchical
- DBSCAN
- Gaussian Mixture Model (GMM)

### Dimensionality Reduction

- PCA
- t-SNE
- UMAP

### Association Rule Mining

- Apriori
- FP-Growth

Goal: Discover hidden structure or patterns in data without labels.

## Python Code Example

```
# Import
from sklearn.cluster import KMeans
import pandas as pd

# Sample data
data = pd.DataFrame({
    'Feature1': [1, 2, 1.5, 5, 5.5, 6, 8, 8.5, 9],
    'Feature2': [1, 2, 1.8, 5, 5.2, 5.8, 8, 8.2, 9.5]
})

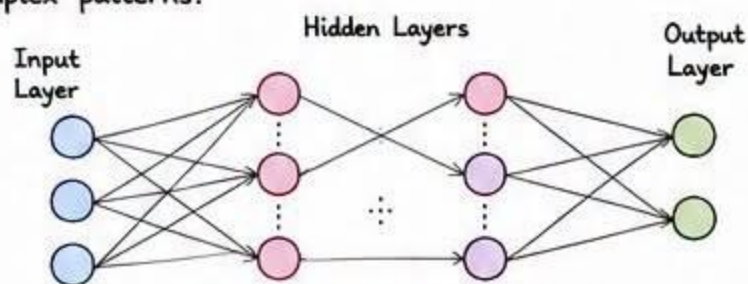
# Model
model = KMeans(n_clusters=3, random_state=42)

# Fit & Get Cluster Labels
model.fit(data)
labels = model.labels_

# Result
print("Cluster Labels:", labels)
```

# 3. DEEP LEARNING

Use neural networks with multiple layers to learn complex patterns.



## Common Use Cases



Image Recognition



Speech Recognition



Natural Language Processing



Recommender Systems



Time Series Forecasting

Goal: Automatically learn hierarchical features from data using neural networks.

## Python Code Example (Keras / TensorFlow)

```
# Import
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np

# Sample data
X = np.array([[1, 2], [2, 3], [3, 4], [4, 5]])
y = np.array([0, 0, 1, 1])

# Model
model = Sequential([
    Dense(8, activation='relu', input_shape=(2,)),
    Dense(4, activation='relu'),
    Dense(1, activation='sigmoid')
])

# Compile
model.compile(optimizer='adam', loss='binary_crossentropy',
              metrics=['accuracy'])

# Train
model.fit(X, y, epochs=100, verbose=0)

# Predict
pred = (model.predict(X) > 0.5).astype(int)
print("Predictions:", pred.flatten())
```



## Quick Import Guide

**Supervised Learning**  
from sklearn.model\_selection import train\_test\_split  
from sklearn.ensemble import RandomForestClassifier

**Unsupervised Learning**  
from sklearn.cluster import KMeans  
import pandas as pd

**Deep Learning**  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dense  
import numpy as np

## How to Call Functions (Simple Way)

- Create an object: model = ModelName()
- Train/Fit the model: model.fit(X\_train, y\_train)
- Predict: y\_pred = model.predict(X\_test)
- Evaluate/Use results as needed

# One Note for ESSENTIAL ML FORMULAS

## 1. LINEAR MODELS

- Linear Regression

$$\hat{y} = w^T x + b$$

- Cost Function

$$J(w, b) = \frac{1}{2n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

- Gradient

$$\frac{\partial J}{\partial w} = \frac{1}{n} X^T (\hat{y} - y), \quad \frac{\partial J}{\partial b} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)$$

- Update Rule (GD)

$$w := w - \alpha \frac{\partial J}{\partial w}, \quad b := b - \alpha \frac{\partial J}{\partial b}$$

- Gradient Descent (General)

$$\theta := \theta - \alpha \nabla_{\theta} J(\theta)$$

( $\alpha$  = learning rate)

- Ridge Regression (L2)

$$J(w) = \text{MSE} + \lambda \sum_{j=1}^p w_j^2$$

- Lasso Regression (L1)

$$J(w) = \text{MSE} + \lambda \sum_{j=1}^p |w_j|$$

## 2. LOGISTIC REGRESSION & MULTICLASS

- Sigmoid Function

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = w^T x + b$$

- Log Loss (Binary Cross-Entropy)

$$J(w, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

- Softmax (Multiclass)

$$P(y = k | x) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad z_k = w_k^T x + b_k$$

- Cross-Entropy Loss (Multiclass)

$$J = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_{ik} \log(\hat{y}_{ik})$$

## 3. NAIVE BAYES

- Bayes' Theorem

$$P(C_k | x) = \frac{P(x | C_k) P(C_k)}{P(x)}$$

- Conditional Independence Assumption

$$P(x | C_k) = \prod_{i=1}^n P(x_i | C_k)$$

- Prediction

$$\hat{C} = \arg \max_k P(C_k) \prod_{i=1}^n P(x_i | C_k)$$

## 4. DECISION TREES

- Gini Impurity

$$\text{Gini}(t) = 1 - \sum_{k=1}^K p_k^2$$

- Entropy

$$H(t) = - \sum_{k=1}^K p_k \log_2 p_k$$

- Information Gain

$$\text{IG}(t, T) = H(t) - \sum_{j=1}^m \frac{N_j}{N} H(j)$$

$N_j$  = # samples in child  $j$ ,  $N$  = # samples in node  $t$

## 5. SUPPORT VECTOR MACHINE (SVM)

- Soft Margin SVM

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

$$\text{s.t. } y_i (w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

- Hinge Loss

$$L_i = \max(0, 1 - y_i (w^T x_i + b))$$

## 6. K-NEAREST NEIGHBORS (KNN)

- Distance (Euclidean)

$$d(x, x') = \sqrt{\sum_{j=1}^p (x_j - x'_j)^2}$$

- Prediction: majority vote (classification)  
or average (regression) of  $K$  nearest neighbors.

## 7. K-MEANS CLUSTERING

- Objective

$$J = \sum_{i=1}^n \sum_{j=1}^K r_{ij} \|x_i - \mu_j\|^2$$

- Update Rules

$$r_{ij} = \begin{cases} 1, & j = \arg \min_i \|x_i - \mu_i\|^2 \\ 0, & \text{otherwise} \end{cases}$$

$$\mu_j = \frac{\sum_{i=1}^n r_{ij} x_i}{\sum_{i=1}^n r_{ij}}$$

## 8. PCA (PRINCIPAL COMPONENT ANALYSIS)

- Covariance Matrix ( $X_c$  = centered data)

$$C = \frac{1}{n-1} X_c^T X_c$$

- Eigen Decomposition

$$C v_i = \lambda_i v_i$$

- Projection (top  $k$  components)

$$Z = X_c W_k, \quad W_k = [v_1, v_2, \dots, v_k]$$

## 9. RANDOM FOREST

- Prediction (Classification)

$$\hat{y} = \text{mode}(h_1(x), h_2(x), \dots, h_T(x))$$

- Prediction (Regression)

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T h_t(x)$$

## 10. GRADIENT BOOSTING

- Model Update

$$F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

$$\text{where } h_m(x_i) \approx - \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}$$

$\eta$  = learning rate (shrinkage)

## 11. LOSS FUNCTIONS (REGRESSION)

- MAE
$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- MSE
$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- RMSE
$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- $R^2$  Score

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

$\bar{y}$  = mean of  $y$

## 12. CONFUSION MATRIX & CLASSIFICATION METRICS

- Confusion Matrix

| Actual \ Predicted | Positive | Negative |
|--------------------|----------|----------|
| Positive           | TP       | FN       |
| Negative           | FP       | TN       |

- Accuracy

$$\frac{TP + TN}{TP + TN + FP + FN}$$

- Precision

$$\frac{TP}{TP + FP}$$

- Recall (Sensitivity)

$$\frac{TP}{TP + FN}$$

- Specificity

$$\frac{TN}{TN + FP}$$

- F1 Score

$$2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\left( \text{or } \frac{2TP}{2TP + FP + FN} \right)$$

## 13. OTHER USEFUL FORMULAS

- Standardization (Z-score)

$$z = \frac{x - \mu}{\sigma}$$

- Mean

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i$$

- Sample Variance

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

- Sample Covariance

$$\text{Cov}(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$



Consistent practice + strong fundamentals = Mastery in ML!

# All Important Formulas for Machine Learning

## ① Linear Regression

Model equation  $\hat{y} = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$

Mean Squared Error (MSE)  $MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$

Gradient descent update  $\beta := \beta - \alpha \frac{\partial J}{\partial \beta}$

$R^2$  (coefficient of determination)  $R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$

Fits a continuous output by minimizing error.

## ② Logistic Regression

Sigmoid function  $\sigma(z) = \frac{1}{1 + e^{-z}}$

Model equation  $p(y = 1 | x) = \sigma(\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p)$

Binary cross-entropy loss  $L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$

Decision rule (default threshold)  $\hat{y} = 1$  if  $p \geq 0.5$ , else 0

Used for classification problems (0 or 1).

## ③ Naive Bayes

Bayes' theorem  $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$

Classification rule  $\hat{y} = \arg \max_c P(x|c)P(c)$

Gaussian likelihood (for continuous features)  $P(x|c) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$

Assumes features are conditionally independent given the class.

## ④ Support Vector Machine (SVM)

Decision function  $f(x) = w^T x + b$

Optimization (hard-margin SVM)  $\min \frac{1}{2} \|w\|^2$   
subject to  $y_i (w^T x_i + b) \geq 1$

Kernel function (e.g., RBF)  $K(x, x') = \exp(-\gamma \|x - x'\|^2)$

Finds the best separating hyperplane between classes.

## ⑤ Decision Tree (Impurity Measures)

Gini impurity  $Gini = 1 - \sum_{i=1}^k p_i^2$

Entropy  $Entropy = - \sum_{i=1}^k p_i \log_2 p_i$

Information gain  $IG = \text{Parent impurity} - \sum_j \frac{n_j}{n} \times \text{Child impurity}_j$

Choose the best split using impurity reduction.

## ⑥ K-Nearest Neighbors (KNN)

Distance (Euclidean)  $d(x, x') = \sqrt{\sum_{i=1}^d (x_i - x'_i)^2}$

Distance (Manhattan)  $d(x, x') = \sum_{i=1}^d |x_i - x'_i|$

Prediction  $\hat{y} = \text{majority class of } K \text{ nearest neighbors (or average for regression)}$

Predicts using the most similar examples (neighbors).

## ⑦ Clustering (K-Means)

Objective function  $J = \sum_{i=1}^n \|x_i - \mu_{c(i)}\|^2$   
(where  $c(i)$  is the cluster assigned to  $x_i$ )

Update centroid  $\mu_k = \frac{1}{|C_k|} \sum_{x \in C_k} x_i$

Distance (Euclidean)  $d(x, \mu_k) = \|x - \mu_k\|$

Groups similar data points together.

## ⑧ Dimensionality Reduction (PCA)

Principal components  $Z = XW$

Covariance matrix (centered data)  $C = \frac{1}{n-1} X_c^T X_c$   
( $X_c = X - \bar{X}$ , i.e., mean-centered data)

Eigen decomposition  $Cw = \lambda w$

Explained variance ratio  $EVR = \frac{\lambda_i}{\sum_j \lambda_j}$

Finds new uncorrelated components that capture the most variance.

## ⑨ Model Evaluation Metrics

Accuracy  $Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$

Precision  $Precision = \frac{TP}{TP + FP}$

Recall (Sensitivity)  $Recall = \frac{TP}{TP + FN}$

F1-score  $F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$

Mean Absolute Error (MAE)  $MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$

Root Mean Squared Error (RMSE)  $RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$

Helps you measure how well your model performs.

## Quick Tips:

- 1 Understand what each formula means (not just memorize).
- 2 Know when to use each algorithm and its assumptions.
- 3 Practice with real datasets (scikit-learn, etc.).
- 4 Focus on interpretation, not just calculation.
- 5 Keep this sheet for quick reference!

Same formulas. Bigger insights!

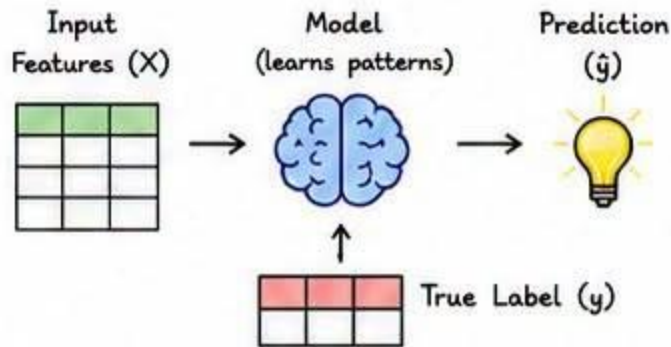


# SUPERVISED LEARNING ALGORITHMS

Learn, Predict, and Make Better Decisions

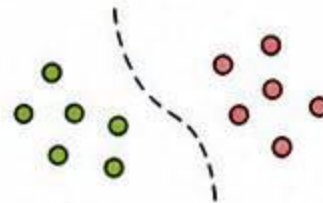
## What is Supervised Learning?

Supervised Learning uses labeled data to learn patterns and make predictions or classifications on new, unseen data.



## How It Works

- 1 Collect labeled data (X, y)
- 2 Choose a model
- 3 Train the model on data
- 4 Evaluate performance
- 5 Use the model to predict new data



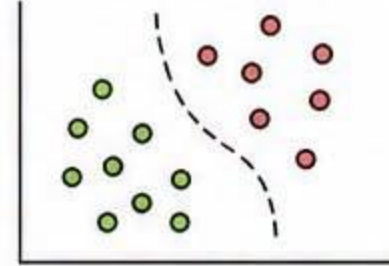
## Types of Supervised Learning

### 1. Classification

Predict a category or class.

Example:

Email is Spam or Not Spam

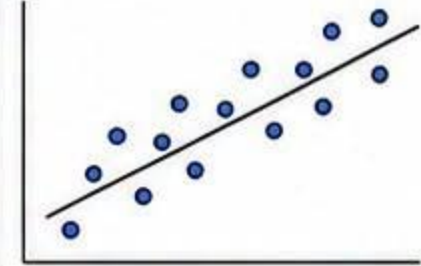


### 2. Regression

Predict a continuous value.

Example:

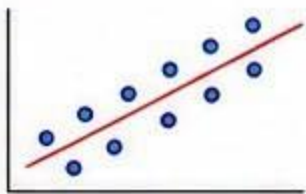
Predict the price of a house



## Popular Supervised Learning Algorithms

### 1. Linear Regression

Models the relationship between features and a continuous target using a straight line.

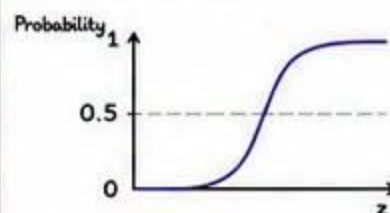


Best for:

- ✓ Continuous values
- ✓ Linear relationships

### 2. Logistic Regression

Predicts the probability of a class using the logistic (S-shaped) function.

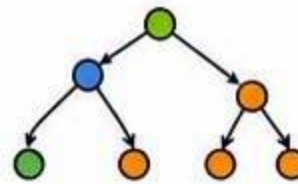


Best for:

- ✓ Binary classification
- ✓ Probabilistic output

### 3. Decision Tree

Splits data based on feature conditions to make decisions.

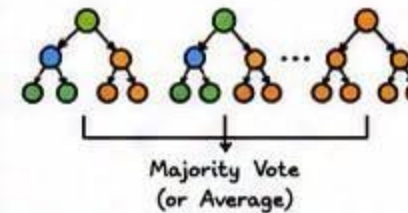


Best for:

- ✓ Easy to interpret
- ✓ Non-linear relationships

### 4. Random Forest

Ensemble of decision trees to reduce variance and capture diverse patterns.

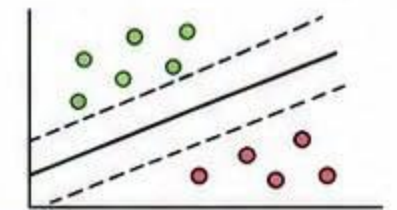


Best for:

- ✓ Reducing variance
- ✓ Handling complex data

### 5. Support Vector Machine (SVM)

Finds the optimal hyperplane that best separates classes with the maximum margin.

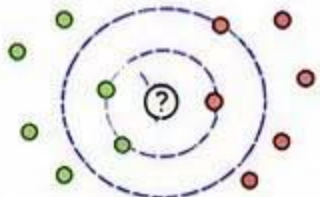


Best for:

- ✓ High-dimensional data
- ✓ Clear margin of separation

### 6. K-Nearest Neighbors (KNN)

Predicts based on the majority class (or average) of the K nearest neighbors.



Best for:

- ✓ Simple and intuitive
- ✓ Small datasets
- ✓ Works well with appropriately scaled features

### 7. Naive Bayes

Uses Bayes' theorem with the assumption of feature independence.

$$P(C|x_1, x_2, \dots, x_n) \propto P(C) \times \prod_{i=1}^n P(x_i|C)$$

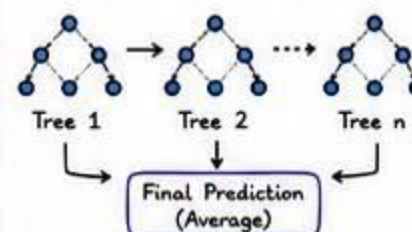
(Proportional to)

Best for:

- ✓ Text classification
- ✓ Fast training
- ✓ Probabilistic prediction

### 8. Gradient Boosting

Builds models sequentially, with each new model reducing the loss of the previous ensemble.



Best for:

- ✓ Complex patterns
- ✓ When strong predictive performance is needed

### 9. XGBoost

Optimized gradient boosting library designed for efficiency and performance.

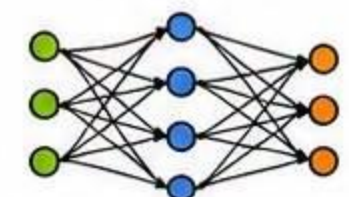


Best for:

- ✓ Structured/tabular data
- ✓ Large-scale datasets

### 10. Neural Networks

Learns patterns through layers of interconnected neurons.



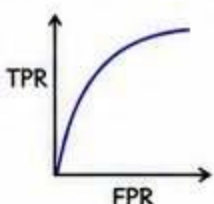
Best for:

- ✓ Complex, non-linear data
- ✓ Deep learning applications

## Evaluation Metrics

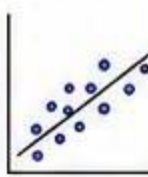
### Classification

- Accuracy
- Precision
- Recall
- F1 score
- ROC-AUC



### Regression

- MAE
- MSE
- RMSE
- R<sup>2</sup> Score



## When to Use Which?

| If your data...                                   | Try this...         |
|---------------------------------------------------|---------------------|
| Linear relationship                               | Linear Regression   |
| Binary classification                             | Logistic Regression |
| Simple, interpretable model                       | Decision Tree       |
| Need a robust tree ensemble                       | Random Forest       |
| Small dataset, local patterns                     | KNN                 |
| Conditional independence is reasonable            | Naive Bayes         |
| Complex nonlinear patterns                        | Gradient Boosting   |
| Structured/tabular data                           | XGBoost / GBM       |
| High-dimensional data                             | SVM                 |
| Complex nonlinear patterns / large feature spaces | Neural Networks     |

## Best Practices

- ✓ Understand your data and problem.
- ✓ Preprocess and clean your data.
- ✓ Split data into Train / Validation / Test.
- ✓ Avoid overfitting, use regularization.
- ✓ Tune hyperparameters.
- ✓ Evaluate using the right metrics.
- ✓ Start simple, then try advanced models.

# REGRESSION ALGORITHMS

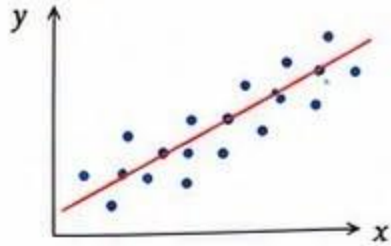
## Predicting Continuous Values

### 1. Simple Linear Regression

Models the relationship between one independent variable and a target.

Model:

$$y = \beta_0 + \beta_1 x + \epsilon$$



**Best for:** Linear relationships with one feature.

**Pros:** Simple, interpretable, fast.

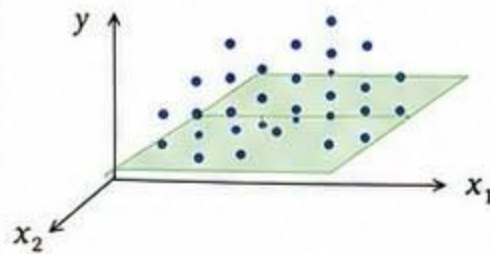
**Cons:** Can underfit complex patterns.

### 2. Multiple Linear Regression

Models the relationship between multiple independent variables and a target.

Model:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$



**Best for:** Linear relationships with multiple features.

**Pros:** Interpretable, works well with linear data.

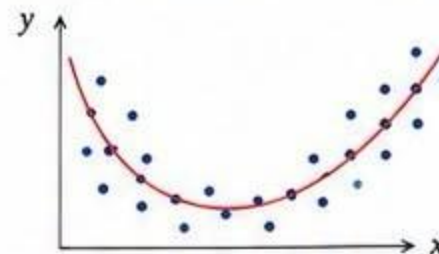
**Cons:** Sensitive to multicollinearity, outliers.

### 3. Polynomial Regression

Captures non-linear relationships by fitting a polynomial equation.

Model:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_n x^n + \epsilon$$



**Best for:** Non-linear relationships (curved patterns).

**Pros:** Simple to implement, captures non-linearity.

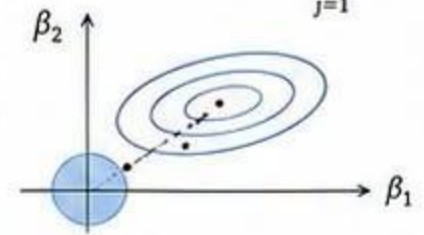
**Cons:** Can overfit with high degree, extrapolates poorly.

### 4. Ridge Regression (L2)

Adds L2 regularization to shrink coefficients and help with multicollinearity.

Objective:

$$\text{Minimize } \text{RSS} + \lambda \sum_{j=1}^n \beta_j^2$$



**Best for:** Datasets with multicollinearity and overfitting.

**Pros:** Shrinks coefficients, helps with multicollinearity, more stable.

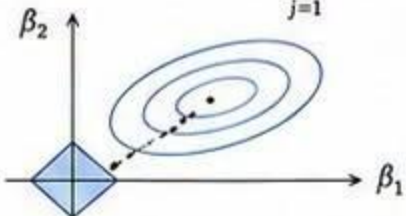
**Cons:** Does not perform feature selection (coefficients rarely become zero).

### 5. Lasso Regression (L1)

Adds L1 regularization that can shrink some coefficients to exactly zero.

Objective:

$$\text{Minimize } \text{RSS} + \lambda \sum_{j=1}^n |\beta_j|$$



**Best for:** Feature selection and sparse models.

**Pros:** Can perform feature selection by shrinking some coefficients to zero.

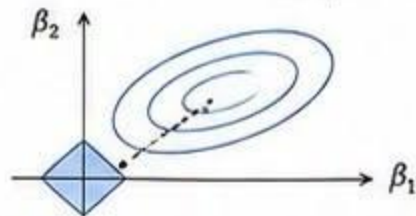
**Cons:** Less stable with correlated features.

### 6. Elastic Net Regression

Combines L1 and L2 regularization to balance Ridge and Lasso.

Objective:

$$\text{Minimize } \text{RSS} + \lambda_1 \sum_{j=1}^n |\beta_j| + \lambda_2 \sum_{j=1}^n \beta_j^2$$



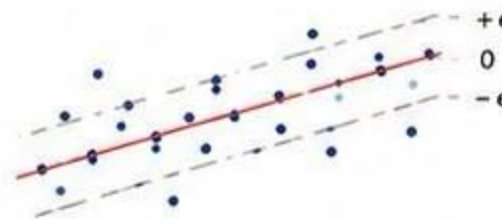
**Best for:** Datasets with correlated features.

**Pros:** Balances Lasso and Ridge, more stable selection.

**Cons:** Requires tuning of two hyperparameters.

### 7. Support Vector Regression (SVR)

Finds a function that deviates from actual targets by a value no more than  $\epsilon$ , while being as flat as possible.



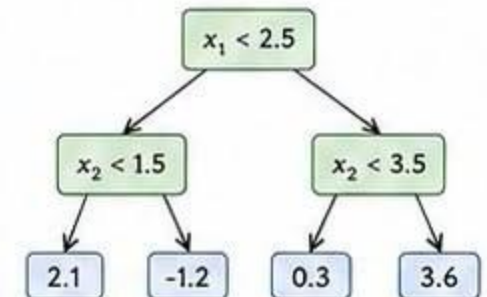
**Best for:** Non-linear relationships, (with appropriate kernels).

**Pros:** Effective in high dimensions, flexible with kernels.

**Cons:** Sensitive to hyperparameters, slow on large datasets.

### 8. Decision Tree Regression

Splits data into regions and predicts the average target value in each region.



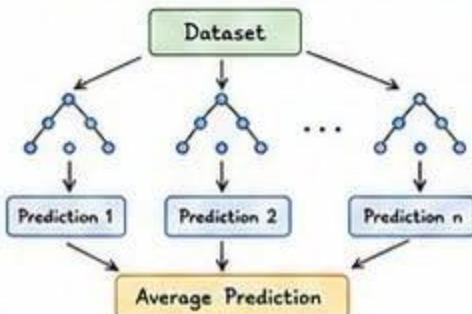
**Best for:** Non-linear relationships and interactions.

**Pros:** Easy to understand, handles mixed data.

**Cons:** Prone to overfitting.

### 9. Random Forest Regression

Ensemble of decision trees. Averages predictions for better accuracy.



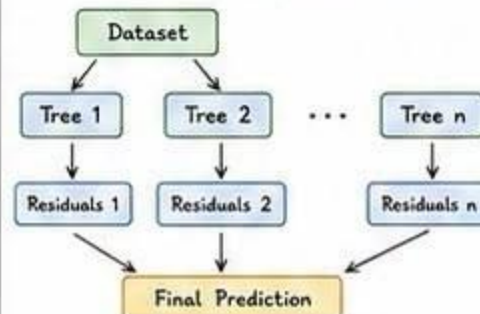
**Best for:** Strong performance on many tabular datasets.

**Pros:** Reduces variance compared with individual trees, works well on large datasets.

**Cons:** Less interpretable, more computational.

### 10. Gradient Boosting Regression

Builds trees sequentially, each fitting the current residuals/errors.



**Best for:** Complex patterns and strong predictive performance.

**Pros:** Very powerful, handles complex patterns.

**Cons:** Sensitive to overfitting, slower to train.

### Evaluation Metrics

$$\text{MAE} = \frac{1}{n} \sum |y_i - \hat{y}_i| \quad \text{Mean Absolute Error}$$

$$\text{MSE} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2 \quad \text{Mean Squared Error}$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2} \quad \text{Root Mean Squared Error}$$

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2} \quad \text{R-squared Score}$$

### When to Use What?

- Linear relationship (1 feature) → Simple Linear Regression
- Linear relationship (many features) → Multiple Linear Regression
- Non-linear patterns → Polynomial / SVR / Tree-based Models
- Many correlated features → Ridge / Elastic Net
- Feature selection needed → Lasso / Elastic Net
- Best predictive performance → Random Forest / Gradient Boosting

### Tips for Better Regression Models

- ✓ Always visualize data before modeling.
- ✓ Handle outliers and missing values.
- ✓ Scale features for regularized models and SVR.
- ✓ Use cross-validation to tune hyperparameters.
- ✓ Evaluate using multiple metrics, not just one.

### Common Libraries in Python

- scikit-learn (sklearn)
- pandas
- numpy
- matplotlib / seaborn
- xgboost / lightgbm / catboost

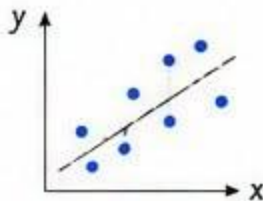
Regression = Understanding Patterns, Making Better Predictions  
The right algorithm + good data = powerful insights!

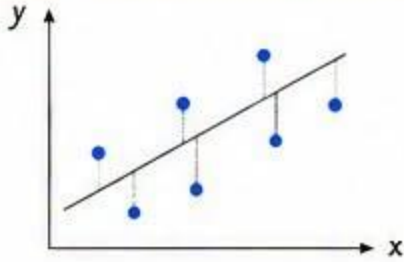
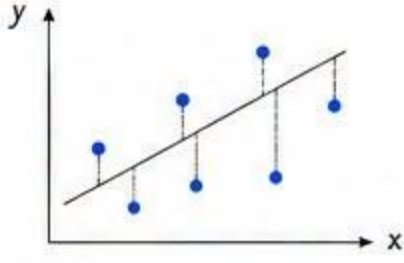
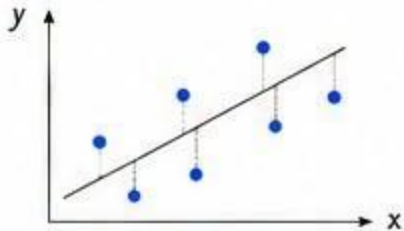
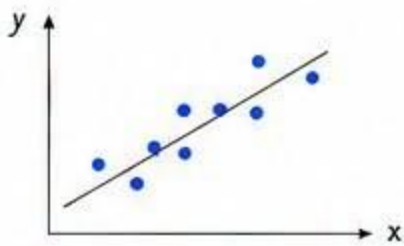
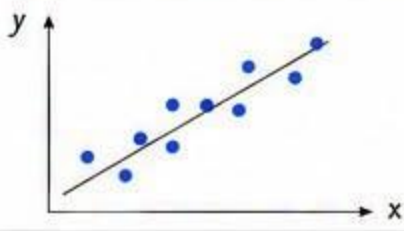
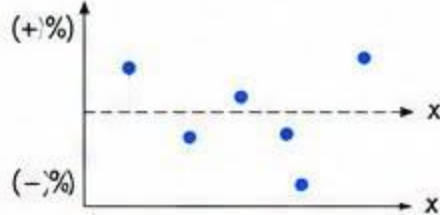
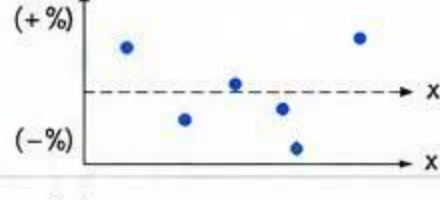
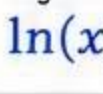
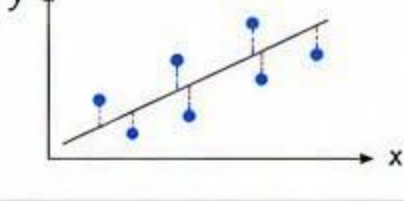
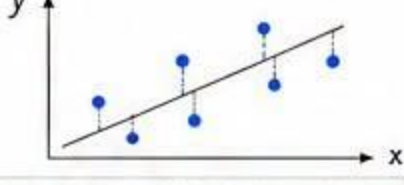


# REGRESSION METRICS

## CHEAT SHEET FOR MACHINE LEARNING

Evaluate how well your model predicts continuous values



| METRIC                                                                                                                                       | FORMULA                                                                                                                                                                             | DESCRIPTION & USE CASE                                                                                                                                                                                                                 | VISUAL EXAMPLE                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>1. MAE</b><br>(Mean Absolute Error)<br>                   | $MAE = \frac{1}{n} \sum_{i=1}^n  y_i - \hat{y}_i $                                                                                                                                  | <ul style="list-style-type: none"><li>Measures average absolute difference between actual and predicted values</li><li>Lower is better</li><li>Less sensitive to outliers</li><li>Use when all errors are equally important</li></ul>  |    |
| <b>2. MSE</b><br>(Mean Squared Error)<br>                    | $MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$                                                                                                                                | <ul style="list-style-type: none"><li>Measures average squared difference between actual and predicted values</li><li>Lower is better</li><li>Penalizes larger errors more</li><li>Sensitive to outliers</li></ul>                     |    |
| <b>3. RMSE</b><br>(Root Mean Squared Error)<br>            | $RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$                                                                                                                        | <ul style="list-style-type: none"><li>Square root of MSE</li><li>Same unit as target variable</li><li>Lower is better</li><li>Easier to interpret than MSE</li></ul>                                                                   |   |
| <b>4. R² Score</b><br>(Coefficient of Determination)<br>   | $R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$                                                                                                 | <ul style="list-style-type: none"><li>Proportion of variance explained by the model</li><li>Range: <math>(-\infty, 1]</math></li><li>1 is perfect prediction</li><li>Can be negative (worse than predicting mean)</li></ul>            |  |
| <b>5. Adjusted R²</b><br>(Adjusted R Squared)<br>          | $\bar{R}^2 = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}$                                                                                                                                 | <ul style="list-style-type: none"><li>Adjusted for number of features (p)</li><li>Penalizes adding unnecessary features</li><li>Higher is better</li><li>Use for model comparison</li><li>Requires <math>n &gt; p + 1</math></li></ul> |  |
| <b>6. MAPE</b><br>(Mean Absolute Percentage Error)<br>     | $MAPE = \frac{100}{n} \sum_{i=1}^n \left  \frac{y_i - \hat{y}_i}{y_i} \right $                                                                                                      | <ul style="list-style-type: none"><li>Average absolute percentage error</li><li>Lower is better</li><li>Easy to interpret (in %)</li><li>Undefined when <math>y_i = 0</math>; unstable near 0</li></ul>                                |  |
| <b>7. WAPE</b><br>(Weighted Absolute Percentage Error)<br> | $WAPE = \frac{\sum_{i=1}^n  y_i - \hat{y}_i }{\sum_{i=1}^n  y_i } \times 100$                                                                                                       | <ul style="list-style-type: none"><li>Uses total absolute error relative to total actual values</li><li>Lower is better</li><li>Can be affected when total actuals are close to zero</li></ul>                                         |  |
| <b>8. SMAPE</b><br>(Symmetric MAPE)<br>                    | $SMAPE = \frac{100}{n} \sum_{i=1}^n \frac{ y_i - \hat{y}_i }{( y_i  +  \hat{y}_i )/2}$                                                                                              | <ul style="list-style-type: none"><li>Symmetric version of MAPE</li><li>Range: <math>[0, 200]\%</math></li><li>Handles zero values better</li><li>Lower is better</li></ul>                                                            |  |
| <b>9. MSLE</b><br>(Mean Squared Logarithmic Error)<br>     | $MSLE = \frac{1}{n} \sum_{i=1}^n (\ln(y_i + 1) - \ln(\hat{y}_i + 1))^2$                                                                                                             | <ul style="list-style-type: none"><li>Penalizes underestimation more than overestimation</li><li>Good for skewed data</li><li>Lower is better</li><li>Requires non-negative actual and predicted values</li></ul>                      |  |
| <b>10. Huber Loss</b><br>                                  | $Huber_{\delta}(e) = \begin{cases} \frac{1}{2} e^2, & \text{if }  e  \leq \delta \\ \delta( e  - \frac{1}{2}\delta), & \text{otherwise} \end{cases}$<br>where $e = y_i - \hat{y}_i$ | <ul style="list-style-type: none"><li>Combines MAE and MSE</li><li>Less sensitive to outliers than MSE</li><li>Controlled by threshold <math>\delta</math></li></ul>                                                                   |  |

**Tip:** Choose metrics based on your problem, data distribution, and business goals. No single metric is perfect for all cases.

### Quick Guide

- Outliers present → Use MAE, Huber, or WAPE
- Interpretability important → Use MAE, RMSE, MAPE
- Model comparison → Use R² or Adjusted R²
- Skewed data → Use MSLE

# K-NEAREST NEIGHBORS (KNN)

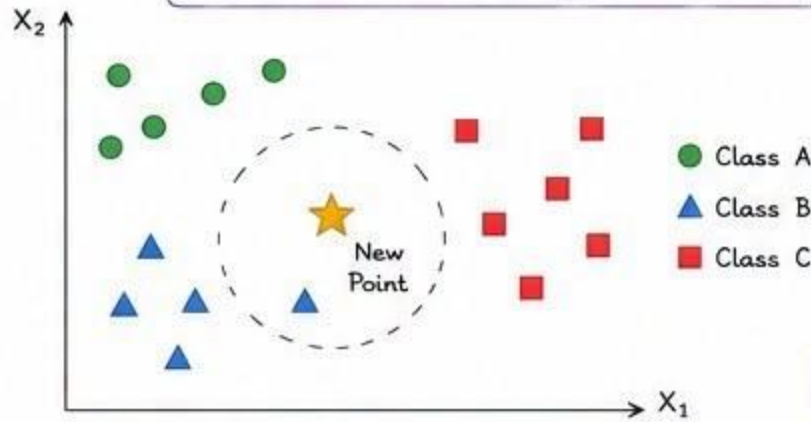
## Using Python

### What is KNN?

K-Nearest Neighbors (KNN) is a simple, easy-to-understand supervised machine learning algorithm.

It can be used for both classification and regression. KNN makes predictions based on the majority vote (classification) or average (regression) of the  $k$  nearest data points.

### 1. How KNN Works (Classification)



1. Choose the value of  $k$  (e.g.,  $k = 5$ ).
2. Calculate the distance from the new point to all points.
3. Select the  $k$  nearest neighbors.
4. Assign the class which appears most frequently among the  $k$  neighbors.

Here,  $k = 5 \rightarrow$  Class B (3 out of 5)

### 2. Distance Metrics

KNN uses distance to find nearest neighbors.

#### • Euclidean Distance

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

#### • Manhattan Distance

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

#### • Minkowski Distance

$$d(x, y) = \left( \sum_{i=1}^n |x_i - y_i|^p \right)^{1/p}$$

( $p=1 \rightarrow$  Manhattan,  $p=2 \rightarrow$  Euclidean)

### 3. Choosing the Right $k$

- Small  $k \rightarrow$  Model is sensitive to noise (overfitting).
- Large  $k \rightarrow$  Model is too general (underfitting).
- Use odd  $k$  for binary classification to avoid ties.

#### Tip:

Use techniques like Cross-Validation to find the best value of  $k$ .

Example: Error vs.  $k$



### 4. KNN Algorithm Steps

1. Choose the value of  $k$ .
2. Calculate the distance between the query point and all training points.
3. Sort the distances and select the  $k$  closest points.
4. For classification: Use majority voting.
5. For regression: Take the average of neighbors.
6. Predict the class or value.

**Note:** Feature scaling is important because KNN is distance-based.

### 5. Pros and Cons

#### Pros

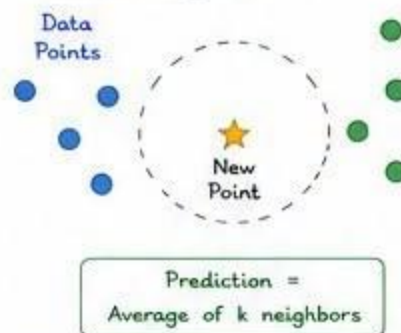
- ✓ Simple and easy to implement
- ✓ No training phase (lazy learner)
- ✓ Works for both classification and regression
- ✓ Non-linear decision boundaries

#### Cons

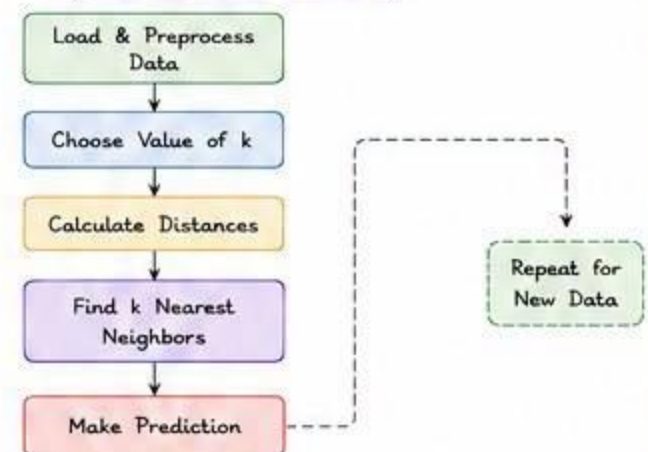
- ✗ Slow prediction for large datasets
- ✗ Sensitive to irrelevant features and scaling
- ✗ Needs optimal  $k$  for good performance
- ✗ High memory usage

### 6. KNN for Regression

Instead of majority voting, KNN regression predicts the average (or weighted average) of the  $k$  nearest neighbors.



### 7. KNN Workflow



### 8. KNN in Python (Scikit-learn)

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report
```

```
# Sample Data (Iris Dataset)
from sklearn.datasets import load_iris
data = load_iris()
X = data.data
y = data.target
```

```
# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

```
# Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# Train KNN Model
k = 5 # Number of neighbors
model = KNeighborsClassifier(n_neighbors=k, metric='minkowski', p=2)
model.fit(X_train, y_train)
```

```
# Predict
y_pred = model.predict(X_test)
```

```
# Evaluate
acc = accuracy_score(y_test, y_pred)
print(f"Accuracy (k={k}): {acc:.2f}")
print(classification_report(y_test, y_pred))
```

#### Explanation

```
# n_neighbors = k
# p=2 -> Euclidean
# p=1 -> Manhattan
# p=3 -> Minkowski
```

```
# Try different
k values to find
the best accuracy!
```



### 9. Example Output

```
Accuracy (k=5): 1.00
Classification Report:
      precision    recall  f1-score   support

0         1.00      1.00      1.00        10
1         1.00      1.00      1.00         9
2         1.00      1.00      1.00        11

 accuracy
macro avg      1.00      1.00      1.00        30
weighted avg    1.00      1.00      1.00        30
```

### 10. Key Points

- KNN is a lazy learning algorithm.
- Choose the right  $k$  for better performance.
- Scale your features before applying KNN.
- Sensitive to the choice of distance metric.
- Works well on small to medium-sized datasets.

#### Common Distance Metrics in KNN

- Euclidean ( $p=2$ ) → Most common
- Manhattan ( $p=1$ ) → Good for sparse data
- Minkowski ( $p=3, 4, \dots$ ) → General form

KNN is simple but powerful when used with the right data and parameters! 💡

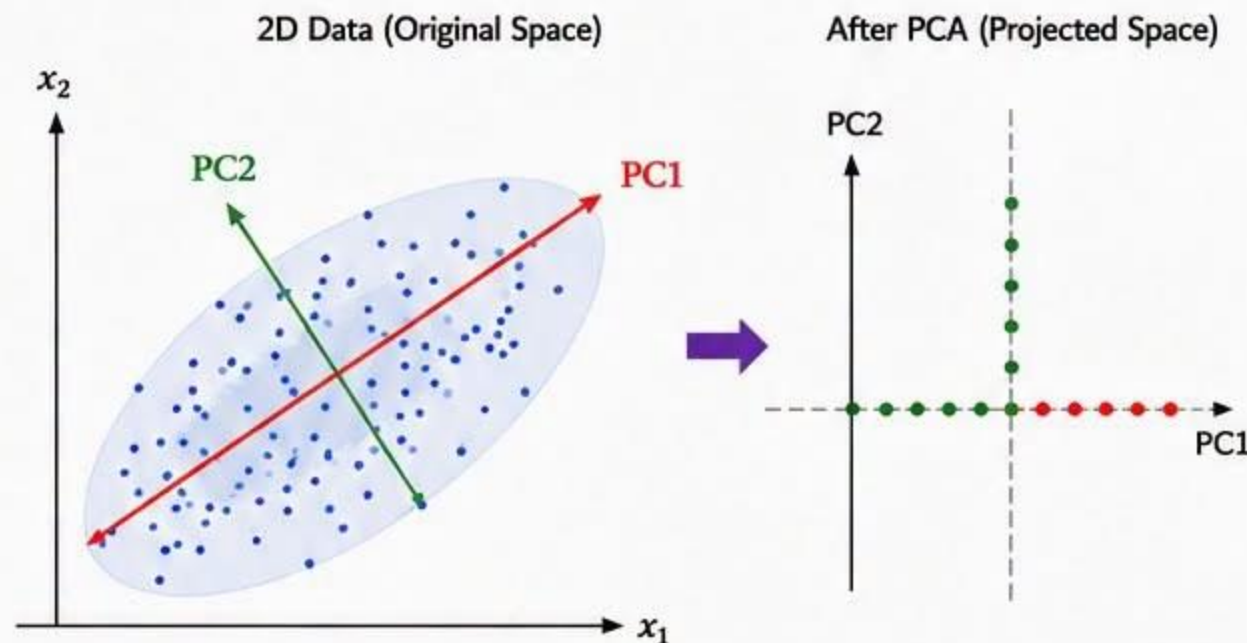
#### Helpful Libraries

NumPy • Pandas • Scikit-learn • Matplotlib

# Principal Component Analysis (PCA)

**Idea:** Transform data into a new coordinate system where the greatest variance lies on the first axis (PC1), the second greatest on the second axis (PC2), and so on.

$X$  : Data matrix ( $n$  samples,  $d$  features)  
 $\bar{X}_c$  : Mean centered data  
 $C$  : Covariance matrix  
 $\lambda_i$  : Eigenvalues (variance along  $PC_i$ )  
 $v_i$  : Eigenvectors (directions of  $PC_i$ )



PC1 captures the maximum variance.  
PC2 captures the remaining variance orthogonal to PC1.

## Key Formulas

1. Mean Centering

$$\bar{X}_c = X - \mu$$

2. Covariance Matrix

$$C = \frac{1}{n-1} \bar{X}_c^T \bar{X}_c$$

3. Eigen Decomposition

$$C v_i = \lambda_i v_i$$

4. Projection (onto  $k$  components)

$$Z = \bar{X}_c V_k$$

( $V_k$  contains top  $k$  eigenvectors)

- Eigenvectors are the principal components.
- Eigenvalues tell us how much variance each principal component captures.
- Projecting data onto top  $k$  components reduces dimensionality while keeping the most important information.

## Key Takeaways

- ➔ PCA finds new orthogonal axes (principal components).
- ➔ PC1 captures the most variance, PC2 the next most, and so on.
- ➔ Helps in dimensionality reduction, noise removal, and visualization.
- ➔ Choose  $k$  components that retain most of the variance.

## Explained Variance Ratio

$$\text{Explained variance by } PC_i = \frac{\lambda_i}{\sum_{j=1}^d \lambda_j}$$

$$\text{Cumulative explained variance} = \sum_{i=1}^k \frac{\lambda_i}{\sum_{j=1}^d \lambda_j}$$



## PCA in Python: Example

Credit: MLTut

```
# PCA Example
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# 1. Load dataset (Iris)
data = load_iris()
X = data.data[:, :2] # use first 2 features for 2D
y = data.target

# 2. Standardize the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# 4. Explained variance
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
print("Cumulative Explained Variance:", np.cumsum(pca.explained_variance_ratio_))

# 5. Principal components (eigenvectors)
print("\nPrincipal Components (Eigenvectors):")
print(pca.components_)

# 6. Plot original vs PCA transformed data
target_names = data.target_names
colors = ['tab:blue', 'tab:green', 'tab:red']

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

# Original data
for i, color in enumerate(colors):
    axes[0].scatter(X[:, 0], X[:, 1], c=color, label=target_names[i], s=30)
axes[0].set_title('Original Data (First 2 Features)')
axes[0].set_xlabel('Feature 1')
axes[0].set_ylabel('Feature 2')
axes[0].legend(frameon=True)
axes[0].grid(True, linestyle='--', alpha=0.3)

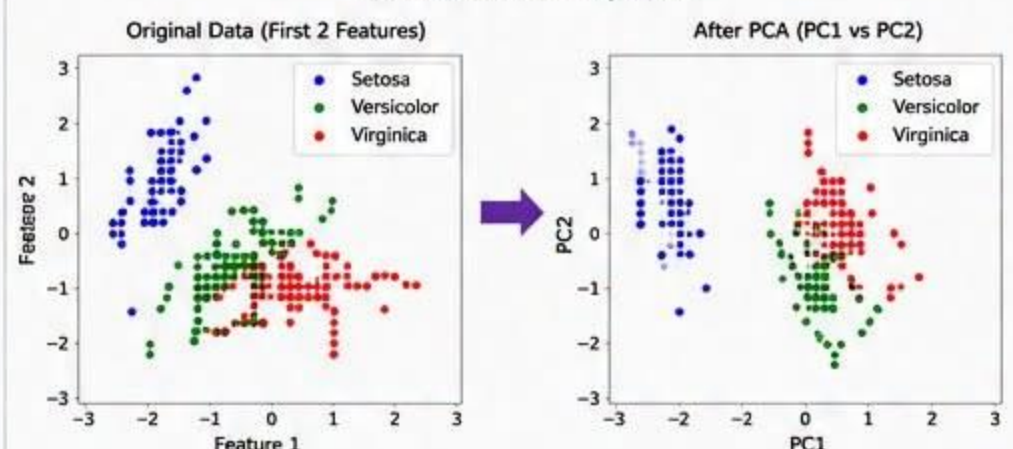
# PCA transformed data
for i, color in enumerate(colors):
    axes[1].scatter(X_pca[:, 0], X_pca[:, 1], c=color, label=target_names[i], s=30)
axes[1].set_title('After PCA (PC1 vs PC2)')
axes[1].set_xlabel('PC1')
axes[1].set_ylabel('PC2')
axes[1].legend(frameon=True)
axes[1].grid(True, linestyle='--', alpha=0.3)

plt.tight_layout()
plt.show()
```

## What the code does

- Loads Iris dataset (2 features for visualization).
- Standardizes the data (mean=0, variance=1).
- Applies PCA to get 2 principal components.
- Prints explained variance and components.
- Plots original vs PCA transformed data.

## PCA Result: 2D Projection



## Output

Explained Variance Ratio: [0.5588 0.4412]  
Cumulative Explained Variance: [0.5588 1.0000]  
Principal Components (Eigenvectors):  
[[ 0.7071 -0.7071]  
 [ 0.7071 0.7071]]

## Interpretation

PC1 explains ~55.88% of the variance.  
PC2 explains ~44.12% of the variance.  
Together they explain 100% of the total variance.

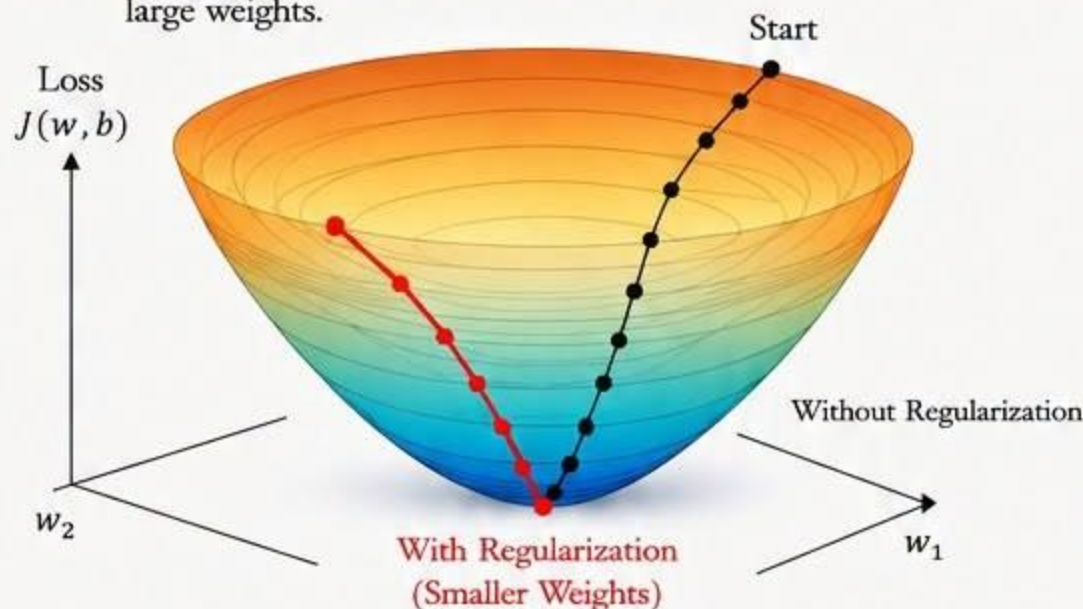


## When to use PCA?

- ✓ Reduce high-dimensional data while preserving important information.
- ✓ Remove noise and redundant features.
- ✓ Improve visualization (2D or 3D plots).
- ✓ Reduce computation in high-dimensional data.

# Regularization in Machine Learning

**Idea:** Add a penalty term to the loss function to prevent overfitting by discouraging large weights.



$w$  : Weight vector  
 $b$  : Bias  
 $\lambda$  : Regularization strength (controls penalty)  
 $J(w, b)$  : Original loss function

## Regularized Loss Function

$$J_{reg}(w, b) = J(w, b) + \lambda \Omega(w)$$

where  $\Omega(w)$  is the penalty term.

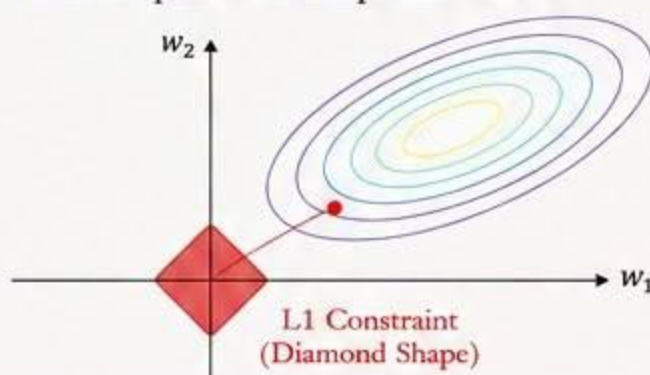
- Adds a penalty for large weights.
- Helps the model generalize better.
- $\lambda$  controls the amount of regularization.
- Too large  $\lambda$  may underfit the model.

## L1 Regularization (Lasso)

Penalty Term:  $\Omega(w) = \sum_{j=1}^m |w_j|$

Loss Function:  $J_{reg}(w, b) = J(w, b) + \lambda \sum_{j=1}^m |w_j|$

- Encourages sparsity (sets some weights exactly to 0).
- Can perform feature selection by driving some coefficients to zero.
- Produces sparse and interpretable models.

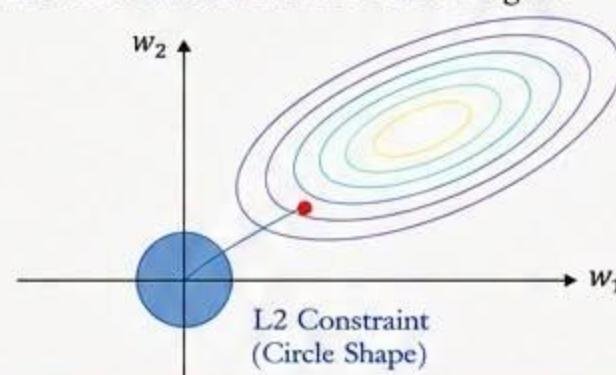


## L2 Regularization (Ridge)

Penalty Term:  $\Omega(w) = \sum_{j=1}^m w_j^2$

Loss Function:  $J_{reg}(w, b) = J(w, b) + \lambda \sum_{j=1}^m w_j^2$

- Shrinks weights towards zero but typically does not make them exactly zero.
- Useful when all features contribute.
- Produces smaller and more stable weights.



## Key Takeaways

- ➔ Regularization adds a penalty to the loss to reduce overfitting.
- ➔ L1 (Lasso) can set some weights to zero.
- ➔ L2 (Ridge) shrinks weights but keeps all features.
- ➔ Choose  $\lambda$  using cross-validation.



Credit: MLTut

## Regularization: Python Code

```
# Regularization in Linear Regression (L1 and L2)
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression, Lasso, Ridge
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

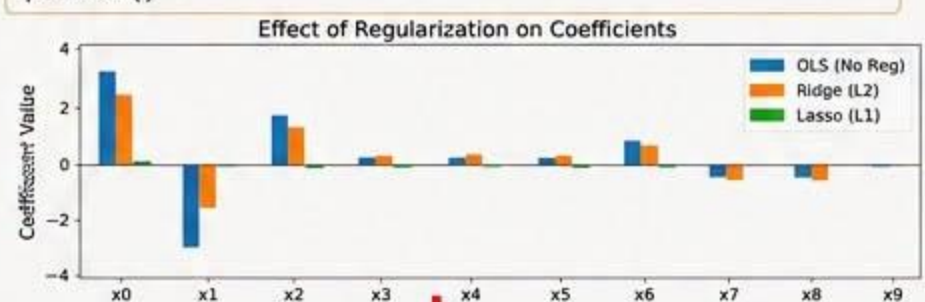
# Generate synthetic data
np.random.seed(42)
X = np.random.randn(100, 10)
true_w = np.array([3, -2, 0, 0, 1.5, 0, 0, 0, 0, 0])
y = X @ true_w + 0.5 * np.random.randn(100)

# Split & standardize
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train models
ols = LinearRegression().fit(X_train, y_train)
ridge = Ridge(alpha=1.0).fit(X_train, y_train)
lasso = Lasso(alpha=0.1).fit(X_train, y_train)

# Coefficients
coef_names = [f'x{i}' for i in range(X.shape[1])]
coef_data = np.vstack([ols.coef_, ridge.coef_, lasso.coef_])
model_names = ['OLS (No Reg)', 'Ridge (L2)', 'Lasso (L1)']
```

```
# Compare coefficient values
x = np.arange(len(coef_names))
width = 0.25
plt.figure(figsize=(10, 4))
plt.bar(x - width, coef_data[0], width, label=model_names[0])
plt.bar(x, coef_data[1], width, label=model_names[1])
plt.bar(x + width, coef_data[2], width, label=model_names[2])
plt.axhline(0, color='black', linewidth=0.8)
plt.xticks(x, coef_names)
plt.xlabel('Coefficient Index')
plt.ylabel('Coefficient Value')
plt.legend()
plt.tight_layout()
plt.show()
```



## Output

Ridge (L2) keeps all coefficients but shrinks them.  
Lasso (L1) drives some coefficients to exactly zero.  
(Close to the data-generating values)

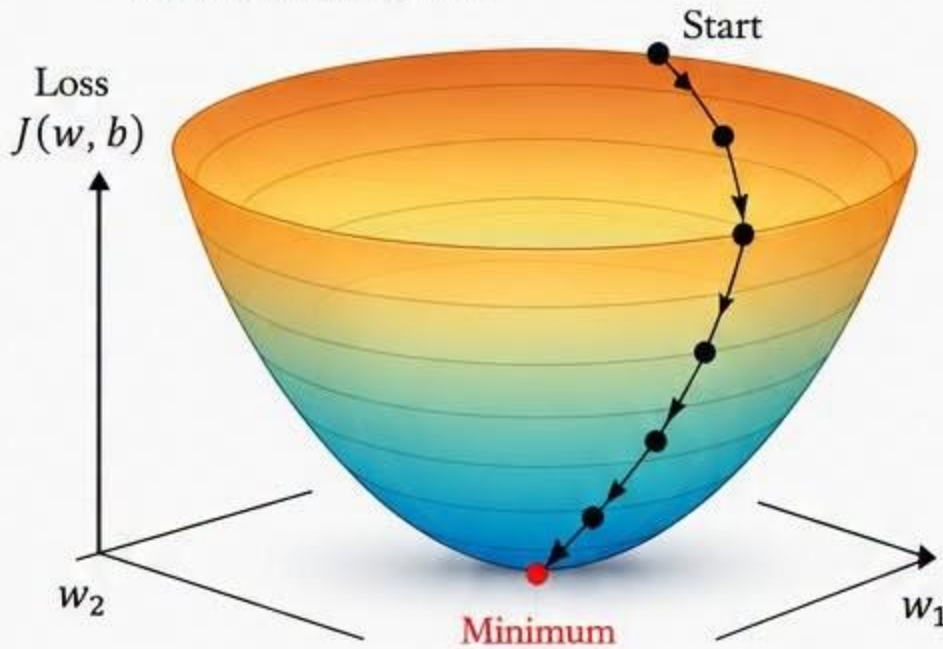
The goal is to find  $w$  and  $b$  that minimize the regularized loss  $J_{reg}(w, b)$ .

$$J_{reg}(w, b) = J(w, b) + \lambda \Omega(w) \quad \text{where } \Omega(w) = \sum_{j=1}^m |w_j| \text{ (L1) or } \sum_{j=1}^m w_j^2 \text{ (L2)}$$

Credit: MLTut

# Gradient Descent

**Idea:** Move step-by-step in the direction that reduces the loss.



$J(w, b)$  : Loss function

$w, b$  : Parameters (weights and bias)

$\nabla J(w, b)$  : Gradient of the loss

## Update Rule

$$w_{new} = w_{old} - \eta \nabla_w J(w_{old}, b_{old})$$

$$b_{new} = b_{old} - \eta \frac{\partial J}{\partial b}(w_{old}, b_{old})$$

- $\eta$  : learning rate (step size)
- The gradient points in the direction of steepest increase of the loss.
- We move in the opposite direction to decrease the loss.

- For a parameter vector  $w \in \mathbb{R}^n$  and bias  $b \in \mathbb{R}$ :

$$w^{(t+1)} = w^{(t)} - \eta \nabla_w J(w^{(t)}, b^{(t)}), \quad b^{(t+1)} = b^{(t)} - \eta \frac{\partial J}{\partial b}(w^{(t)}, b^{(t)})$$

- For linear regression (MSE loss):  $J(w, b) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})^2$ ,  $\hat{y} = Xw + b$

- Gradient w.r.t. weights:  $\nabla_w J(w, b) = \frac{1}{m} X^T (Xw + b - y)$

- Gradient w.r.t. bias:  $\frac{\partial J}{\partial b}(w, b) = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})$

## Key Takeaways

- ➔ Gradient Descent iteratively updates parameters to minimize the loss.
- ➔ The learning rate  $\eta$  controls the step size.
- ➔ Too large  $\eta$  may overshoot, too small  $\eta$  makes learning very slow.

Credit: MLTut

# Gradient Descent: Python Code

```
# Gradient Descent for Linear Regression
import numpy as np

# Generate some data
np.random.seed(0)
X = 2 * np.random.rand(100, 1)
y = 4 * X + 1 + np.random.randn(100, 1) * 0.5 # y = 4x + 1 + noise

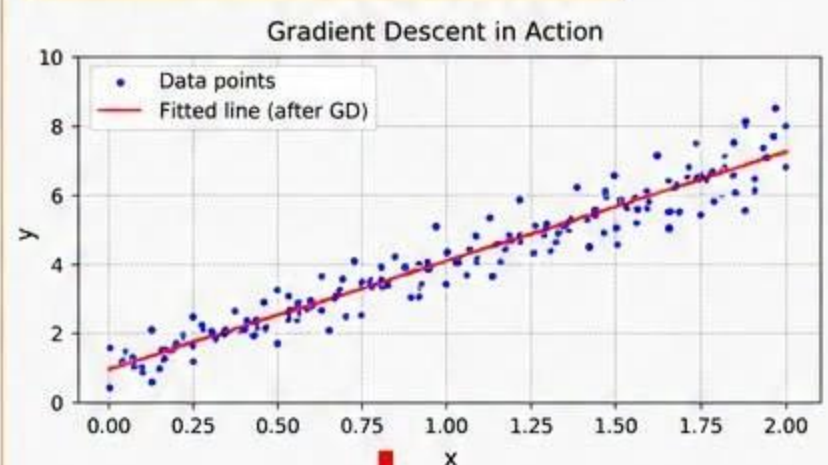
# Initialize parameters
w = np.random.randn(1, 1)
b = np.zeros((1,))
lr = 0.1 # Learning rate
epochs = 100

# Gradient Descent
m = X.shape[0]
for _ in range(epochs):
    y_pred = X @ w + b # predictions
    dw = (1/m) * X.T @ (y_pred - y) # gradient w.r.t w
    db = (1/m) * np.sum(y_pred - y, axis=0) # gradient w.r.t b

    w = w - lr * dw # update weights
    b = b - lr * db # update bias

# Results
print(f'Weight (w): {w[0][0]:.4f}')
print(f'Bias (b): {b[0]:.4f}')
```

```
# Example
# True Line: y = 4x + 1
# Estimated Line after training
x_line = np.linspace(0, 2, 100)
y_line = x_line * w[0][0] + b[0]
```



## Output

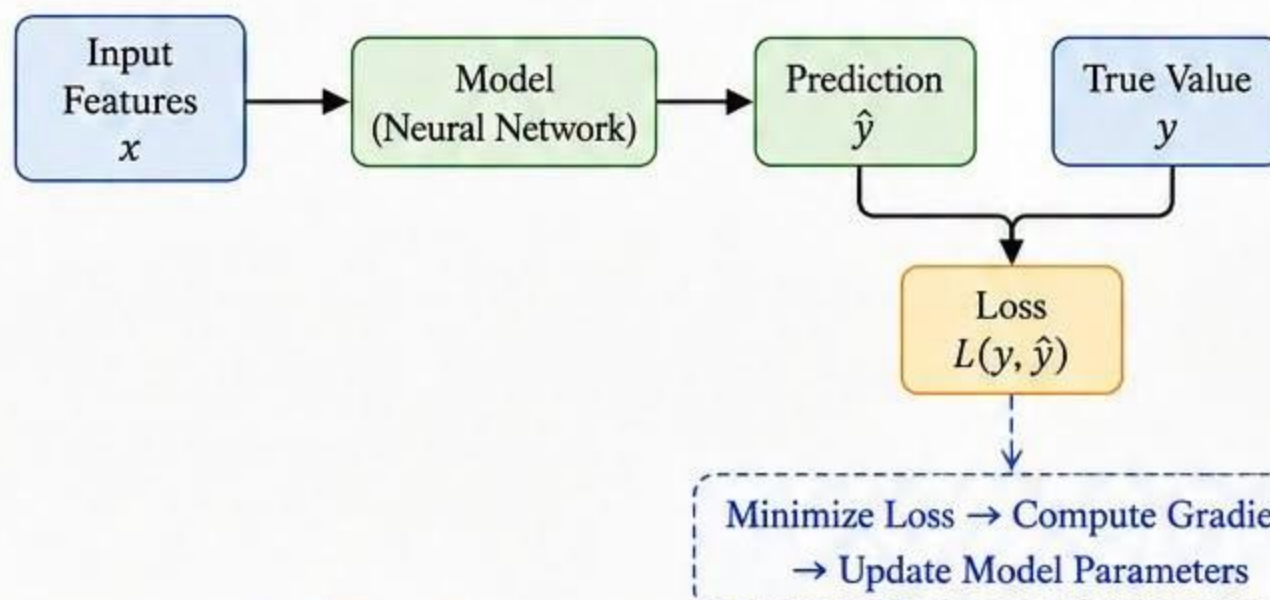
Weight (w): 3.7159  
Bias (b): 1.4135  
(Close to the data-generating values:  $w=4$ ,  $b=1$ )

The goal is to find  $w$  and  $b$  that minimize the loss  $J(w, b)$ .

# Loss Functions



**Loss function** measures how well a model's predictions match the true values.



- $y$  : True value
- $\hat{y}$  : Predicted value
- $L$  : Loss
- $n$  : Number of samples
- $i$  :  $i^{th}$  sample
- $c$  :  $c^{th}$  class
- $C$  : Number of classes

## Common Loss Functions

|                                    |                                                                                             |                                                                                                            |
|------------------------------------|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| 1. Mean Squared Error (MSE)        | $L_{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$                                    | Used for regression problems.<br>Penalizes larger errors more heavily.                                     |
| 2. Mean Absolute Error (MAE)       | $L_{MAE} = \frac{1}{n} \sum_{i=1}^n  y_i - \hat{y}_i $                                      | Used for regression problems.<br>Less sensitive to outliers.                                               |
| 3. Binary Cross-Entropy (BCE)      | $L_{BCE} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$ | Used for binary classification.<br>Measures how well predicted probabilities match the true binary labels. |
| 4. Categorical Cross-Entropy (CCE) | $L_{CCE} = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C y_{ic} \log(\hat{y}_{ic})$                | Used for multi-class classification.<br>$C$ = Number of classes.                                           |

Credit: MLTut

## Loss Functions: Python Code

```
# Import required module
import numpy as np

# True values and predicted values
# Regression example
y_true_reg = np.array([3.0, -0.5, 2.0, 7.0])
y_pred_reg = np.array([2.5, 0.0, 2.5, 6.0])

# Binary classification (0 or 1)
y_true_bin = np.array([1, 0, 1, 1, 0])
y_pred_bin = np.array([0.9, 0.2, 0.8, 0.7, 0.1])

# Multi-class classification (3 classes)
y_true_multi = np.array([
    [1, 0, 0],
    [0, 1, 0],
    [0, 0, 1],
    [1, 0, 0]
])
y_pred_multi = np.array([
    [0.7, 0.2, 0.1],
    [0.1, 0.8, 0.1],
    [0.2, 0.3, 0.5],
    [0.6, 0.3, 0.1]
```

```
# Loss Functions
def mse(y, y_hat):
    return np.mean((y - y_hat) ** 2)

def mae(y, y_hat):
    return np.mean(np.abs(y - y_hat))

def bce(y, y_hat):
    eps = 1e-15 # To avoid log(0)
    y_hat = np.clip(y_hat, eps, 1 - eps)
    return -np.mean(y * np.log(y_hat) +
                    (1 - y) * np.log(1 - y_hat))

def cce(y, y_hat):
    eps = 1e-15
    y_hat = np.clip(y_hat, eps, 1 - eps)
    return -np.mean(np.sum(y * np.log(y_hat), axis=1))

# Calculate Losses
print(f'MSE Loss: {mse(y_true_reg, y_pred_reg):.4f}!')
print(f'MAE Loss: {mae(y_true_reg, y_pred_reg):.4f}!')
print(f'BCE Loss: {bce(y_true_bin, y_pred_bin):.4f}!')
print(f'CCE Loss: {cce(y_true_multi, y_pred_multi):.4f}!')
```

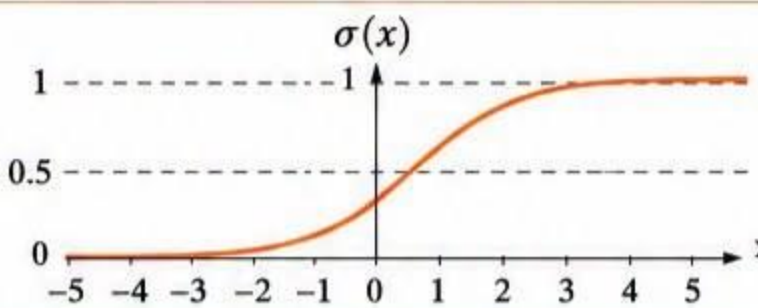
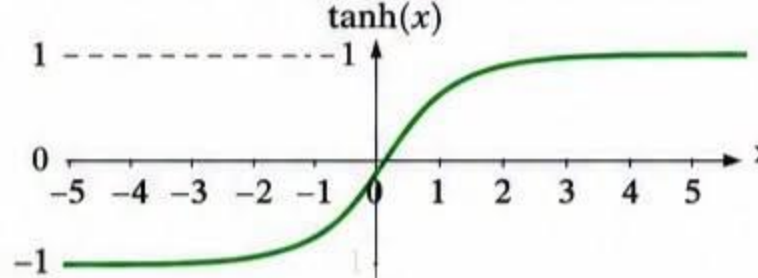
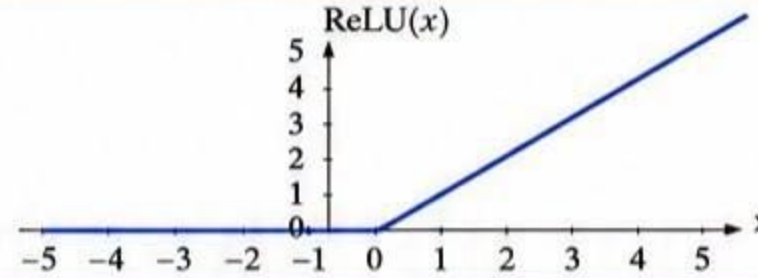
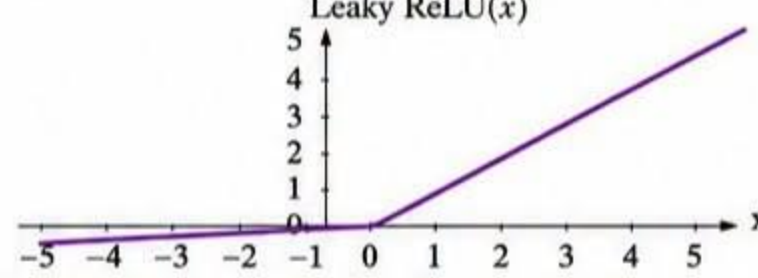
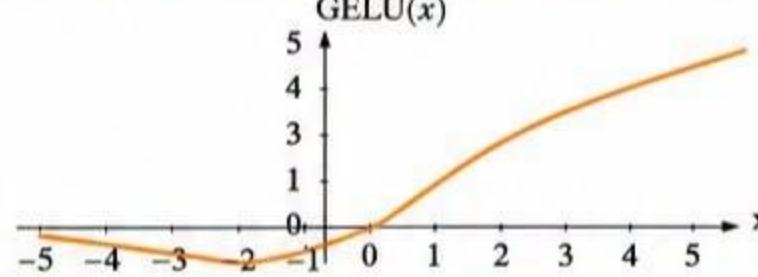
**Output:**

```
MSE Loss: 0.4375
MAE Loss: 0.6250
BCE Loss: 0.2027
CCE Loss: 0.4459
```

Credit: MLTut

# Activation Functions

Activation functions introduce non-linearity in neural networks, enabling the model to learn complex patterns.

| Function                                                  | Formula                                                                                                                                                                                                                                                                                                              | Graph ( $x \in [-5, 5]$ )                                                             |
|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Sigmoid</b><br>Output range:<br>$(0, 1)$               | $\sigma(x) = \frac{1}{1 + e^{-x}}$ <ul style="list-style-type: none"> <li>Smooth S-shaped curve</li> <li>Used in binary classification</li> </ul>                                                                                                                                                                    |    |
| <b>Tanh</b><br>Output range:<br>$(-1, 1)$                 | $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ <ul style="list-style-type: none"> <li>Zero-centered</li> <li>Often used in hidden layers</li> </ul>                                                                                                                                                                  |    |
| <b>ReLU</b><br>Output range:<br>$[0, \infty)$             | $\text{ReLU}(x) = \max(0, x)$ <ul style="list-style-type: none"> <li>Most popular activation</li> <li>Fast and simple</li> </ul>                                                                                                                                                                                     |   |
| <b>Leaky ReLU</b><br>Output range:<br>$(-\infty, \infty)$ | $\text{Leaky ReLU}(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$ <ul style="list-style-type: none"> <li>Fixes "dying ReLU" problem</li> <li><math>\alpha</math> is a small constant (e.g., 0.01)</li> </ul>                                                                                     |  |
| <b>GELU</b><br>Output range:<br>$(-\infty, \infty)$       | $\text{GELU}(x) = x \times \frac{1}{2} \left( 1 + \text{erf} \left( \frac{x}{\sqrt{2}} \right) \right)$ <p>where <math>\text{erf}(z) = \frac{2}{\sqrt{\pi}} \int_0^z e^{-t^2} dt</math></p> <ul style="list-style-type: none"> <li>Smooth, non-monotonic</li> <li>Widely used in Transformer-based models</li> </ul> |  |

## Python Implementation

```
import numpy as np
from scipy.special import erf
# Activation functions
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
def tanh(x):
    return np.tanh(x)
def relu(x):
    return np.maximum(0, x)
def leaky_relu(x, alpha=0.01):
    return np.where(x > 0, x, alpha * x)
def gelu(x):
    return 0.5 * x * (1 + erf(x / np.sqrt(2)))
```

### Tips

- ReLU is fast but can suffer from dying neurons.
- Leaky ReLU mitigates dying neurons.
- Tanh is zero-centered, good for RNNs.
- GELU is commonly used in Transformer-based models.

## Example

```
x = np.linspace(-3, 3, 7)
print('Input:\n', x)
print('Sigmoid:\n', sigmoid(x))
print('Tanh:\n', tanh(x))
print('ReLU:\n', relu(x))
print('Leaky ReLU (alpha=0.01):\n', leaky_relu(x))
print('GELU:\n', gelu(x))
```



```
Input:
[-3. -2. -1.  0.  1.  2.  3.]
Sigmoid:
[0.0474 0.1192 0.2689 0.5  0.7311 0.8808 0.9526]
Tanh:
[-0.9951 -0.9640 -0.7616  0.  0.7616  0.9640  0.9951]
ReLU:
[0.  0.  0.  0.  1.  2.  3.]
Leaky ReLU (alpha=0.01):
[-0.03 -0.02 -0.01  0.  1.  2.  3.]
GELU:
[-0.0040 -0.0455 -0.1587  0.  0.8413  1.9545  2.9960]
```

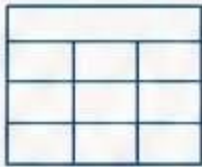
Activation functions help neural networks learn  
**non-linear relationships.**

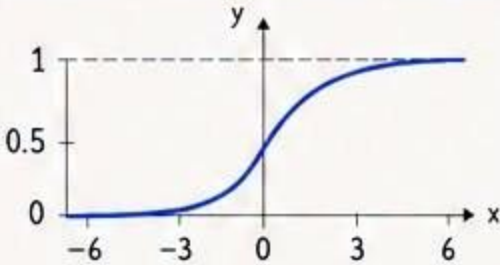
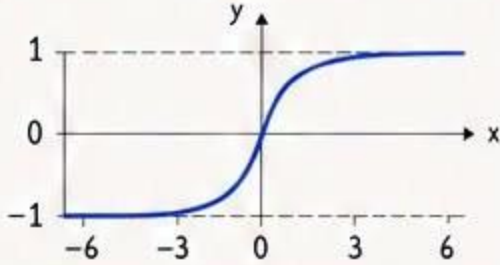
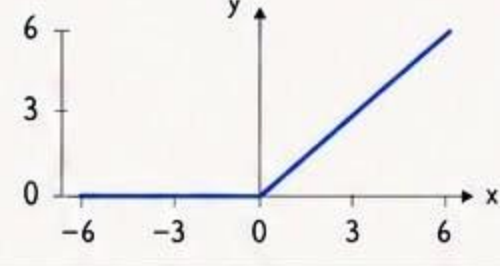
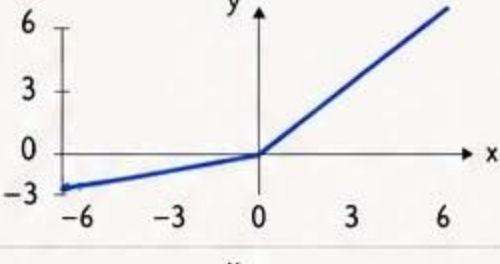
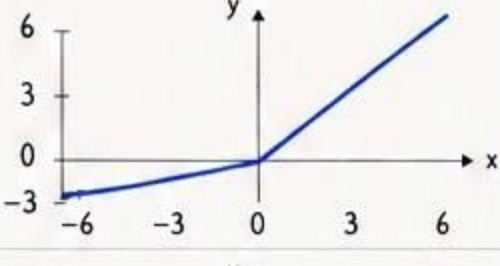
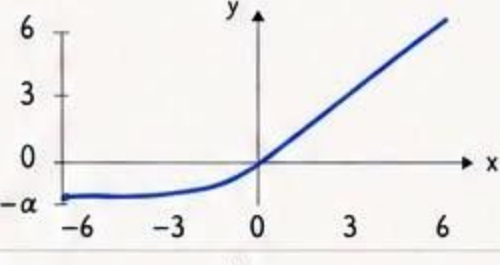
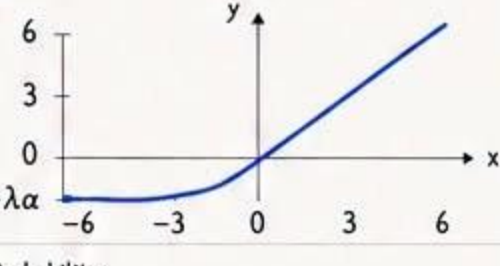
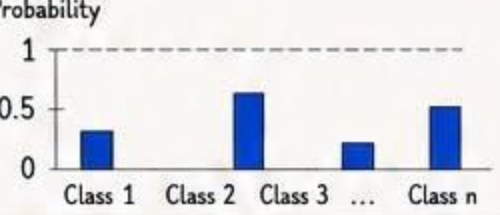
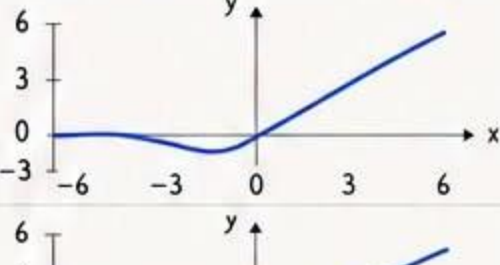
# ACTIVATION FUNCTIONS

## CHEAT SHEET FOR DEEP LEARNING

Quick Import

from tensorflow.keras  
import activations



| ACTIVATION FUNCTION                                                                              | FORMULA                                                                                      | DESCRIPTION & USE CASE                                                                                                                                                               | VISUAL EXAMPLE                                                                        |
|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
|  1. Sigmoid      | $f(x) = \frac{1}{1 + e^{-x}}$                                                                | <ul style="list-style-type: none"><li>Output range: (0, 1)</li><li>Used in binary classification</li><li>Not zero-centered</li><li>Can suffer from vanishing gradients</li></ul>     |    |
|  2. Tanh         | $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$                                                   | <ul style="list-style-type: none"><li>Output range: (-1, 1)</li><li>Zero-centered</li><li>Common in hidden layers</li><li>Can suffer from vanishing gradient</li></ul>               |    |
|  3. ReLU       | $f(x) = \max(0, x)$                                                                          | <ul style="list-style-type: none"><li>Output range: [0, ∞)</li><li>Most popular activation</li><li>Fast convergence</li><li>Can suffer from the dying ReLU problem (x ≤ 0)</li></ul> |   |
|  4. Leaky ReLU | $f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$<br>(α ≈ 0.01)          | <ul style="list-style-type: none"><li>Output range: (-∞, ∞)</li><li>Reduces the dying ReLU problem</li><li>Small slope for x &lt; 0</li></ul>                                        |  |
|  5. PReLU      | $f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$<br>(α is learnable)    | <ul style="list-style-type: none"><li>Output range: (-∞, ∞)</li><li>Parametric version of Leaky ReLU</li><li>α is a learnable parameter</li></ul>                                    |  |
|  6. ELU        | $f(x) = \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}$<br>(α > 0)      | <ul style="list-style-type: none"><li>Output range: (-α, ∞)</li><li>Smooth at x = 0</li><li>Helps learning by shifting activations closer to zero mean</li></ul>                     |  |
|  7. SELU       | $f(x) = \begin{cases} \lambda x, & x > 0 \\ \lambda \alpha(e^x - 1), & x \leq 0 \end{cases}$ | <ul style="list-style-type: none"><li>Output range: (-λ α, ∞)</li><li>Enables self-normalizing networks</li><li>Maintains activations close to zero mean and unit variance</li></ul> |  |
|  8. Softmax    | $f_i(x) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$                                              | <ul style="list-style-type: none"><li>Output range: (0, 1)</li><li>Sum of outputs = 1</li><li>Used in multi-class classification (output layer)</li></ul>                            |  |
|  9. Swish      | $f(x) = x \cdot \sigma(x)$<br>(σ = Sigmoid.)                                                 | <ul style="list-style-type: none"><li>Output range: (-∞, ∞)</li><li>Smooth and non-monotonic</li><li>Often outperforms ReLU in some deep neural networks</li></ul>                   |  |
|  10. GELU      | $f(x) = 0.5x \left( 1 + \tanh \left[ \sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right] \right)$ | <ul style="list-style-type: none"><li>Output range: (-∞, ∞)</li><li>Widely used in Transformer models (e.g., BERT, GPT)</li><li>Smooth version of ReLU</li></ul>                     |  |

**Tip:** Choose activation based on your problem, data, and network type.  
ReLU family → default choice for hidden layers.

Common Choices

Hidden Layers : ReLU / Leaky ReLU / PReLU / GELU / Swish

Output Layer : Sigmoid (Binary) / Softmax (Multi-class)